Cluster Computing (2024) 27:6289–6304
https://doi.org/10.1007/s10586-024-04349-y

A distributed intelligence framework for enhancing resilience and data privacy in dynamic cyber-physical systems

Nabila Azeri[1] · Ouided Hioual[2] · Ouassila Hioual[2,3]

Received: 27 October 2023 / Revised: 4 February 2024 / Accepted: 7 February 2024 / Published online: 27 February 2024
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract
Cyber-Physical Systems (CPS) are integral components of modern, interconnected environments, playing a crucial role in various applications such as smart infrastructure and autonomous systems. These systems operate in complex and ever-evolving settings, where resilience is an essential characteristic. Recently, several Machine Learning (ML) techniques have been proposed to design and implement resilience in CPS. However, this approach has often prioritized performance gains and adaptability enhancements, leading to a disregard for the potential dangers linked to centralized data processing, including data breaches and privacy infringements. To harness the full potential of ML in CPS, this paper introduces a distributed intelligence framework that equally prioritizes security, data privacy, and adaptability. The proposed framework is implemented through the integration of Federated Machine Learning techniques, where the CPS architecture is decentralized, allowing data processing to occur locally on individual nodes or devices. This decentralized approach facilitates the aggregation of insights from multiple sources without the need for centralized data processing, thereby minimizing the risks associated with data breaches and privacy violations. We further validate the viability of the proposed framework through its successful implementation in a real-world industrial CPS application, specifically focused on fault prediction within industrial CPS environments. In addition to its privacy and security benefits, our approach also achieved promising results in terms of accuracy and precision.

Keywords Cyber-physical systems · Multi-layer architecture · Federated machine learning · Fault prediction

1 Introduction

In the ever-evolving landscape of technology, Cyber-Physical Systems (CPS) have emerged as a crucial aspect of our interconnected world, seamlessly merging the physical and digital realms [25]. These systems, which encompass everything from autonomous vehicles and industrial automation to smart cities and healthcare applications, are characterized by their ability to interact with the physical environment, process data, and make intelligent decisions in real-time [1, 27]. However, this growing demand raises a challenge: how to ensure that these systems not only meet their design requirements, but also adapt and evolve in response to rapid environmental changes.

The need for adaptation within CPS stems from their inherently intricate nature. Unlike traditional systems, CPS exhibit a high degree of heterogeneity, involving diverse components that operate at different levels of abstraction. Moreover, the dynamic and often unpredictable nature of the physical world adds an additional layer of complexity. As a result, the design and development of CPS that can effectively navigate this complexity and remain in

✉ Nabila Azeri
azeri.nabila@univ-khenchela.dz

Ouided Hioual
hioual.ouided@univ-khenchela.dz

Ouassila Hioual
hioual_ouassila@univ-khenchela.dz

[1] ICOSI Laboratory, Abbes Laghrour University, BP 1252 Road of Batna, El Hamma, 40004 Khenchela, Algeria

[2] Mathematics and Informatics Department, Abbes Laghrour University, BP 1252 Road of Batna, El Hamma, 40004 Khenchela, Algeria

[3] LIRE Laboratory, Constantine 2 University, BP67A, Ali Mendjeli, 25000 Constantine, Algeria

compliance with requirements have become a principal concern [10, 22].

Driven by the need for adaptable CPS, the integration of Machine Learning (ML) has become a prominent focus within relevant research and development communities. The inherent benefits of ML, such as its ability to process vast amounts of data, learn from experience, and make real-time predictions, make it ideal for ensuring rapid adaptation in these dynamic systems [2, 7]. Numerous studies have explored the utilization of ML techniques to develop flexible and adaptive CPS capable of responding to changing environmental conditions. These techniques often involve a central storage of data, followed by the training of a model on this centralized data repository. This approach facilitates comprehensive analysis and decision-making on a global scale. However, this centralized approach gives rise to potential challenges concerning data privacy and security, as all the information is consolidated in a single location [18, 24].

This situation calls for the emergence of Federated Machine Learning (FML), a methodology that embraces a decentralized perspective while retaining data on local devices or servers, such as mobile phones or IoT devices. In this paradigm, the model is disseminated and trained locally on these distributed datasets, eliminating the necessity of transferring data to a central repository [30]. By distributing the computational load across multiple devices, FML can overcome the limitations of centralized processing in CPS. It harnesses the distributed computing capabilities of CPS systems, allowing efficient processing and analysis of large volumes of data.

This research proposes a decentralized multi-layer architecture for resilient CPS by combining the agility and low-latency capabilities of Edge computing [15] with the scalability and centralized processing power of Cloud computing [28]. The FML facilitates the seamless interaction between Edge and Cloud components, enabling distributed intelligence and efficient decision-making while preserving data privacy and security. This work serves as an extension and progression of our previous investigations, wherein we delved into the CPS paradigm's self-adaptive facet through the conceptualization of a meta-model designed to underpin resilient CPS [3]. Furthermore, our research introduces a centralized architectural tailored to the unique requirements of CPS. The proposed architecture contains the main software modules that enhance CPS to be self-aware, self-learning and self-reconfiguring. A focal point of our architectural is the strategic integration of ML algorithms which can lead to systems that are highly adaptable, efficient, and capable of addressing dynamic changes [4]. By embracing the FML paradigm, we seek in this paper to profoundly enhance the adaptive capabilities of CPS, and improve our previous work in term of security and data privacy while trying to maintain the accuracy and precision of the learning model. More precisely, this paper makes the following contributions:

- Proposing a decentralized multi-layer architecture: Introducing a novel architecture that combines the agility of Edge computing with the scalability and centralized processing power of Cloud computing to enhance the resilience of CPS.
- Integrating FML in our architecture: Our proposed architecture integrates FML, emphasizing its role in enabling decentralized training and inference processes. This integration ensures not only the efficiency of the system but also addresses concerns related to data privacy and security.
- Real-world application and implementation: Demonstrating the practical application of the proposed architecture on a real-world example, specifically focusing on fault prediction of a system encountered in the industry.

The rest of the paper is structured as follows. Section 2 presents some related work. Section 3 introduces our previous work concerning meta-models and a centralized architecture. Section 4 describes the proposed FML-based architecture. The implementation and application of our architecture on a real-world example are detailed in Sect. 5. Finally, some concluding remarks and directions for future work are given in Sect. 6.

2 Related work

In recent years, ML techniques have gained significant attention as a means to enhance the adaptability and efficiency of CPS. Several studies have explored the integration of ML into CPS to address the challenges posed by dynamic environments and changing user requirements. Many research efforts have focused on centralized ML solutions within CPS. These approaches leverage centralized data processing and model training, pooling data from various sources to build global models.

Table 1 provides an overview of relevant research using ML techniques to enhance resilience and security in CPS. For instance, in [9], authors propose a complete real-world CPS implementation cycle, ranging from machine data acquisition to processing and interpretation. In fact, they propose a CPS for machine component knowledge discovery based on clustering algorithms using real data from a machining process. More precisely, they propose the development of a component behavior multidimensional pattern using machine learning clustering techniques. Clustering algorithms aim to partition a centralized dataset into clusters, interpreted as behavior patterns. Some

Table 1 Overview of ML-based approaches for enhancing resilience and security in CPS

Papers	Goal	Method ML	Learning process	Privacy considerations	Security considerations	Commu. overhead
Diaz et al. [9]	Machine component knowledge discovery	Clustering algorithms based on real data	Centralized	No	No	High
Wu et al. [31]	Detecting Cyber Physical attacks	Image classification using ML algorithms	Centralized	No	No	High
Babouche et al. [5]	Self-adaptive and scalable prediction/detection mechanism	Multiple ML algorithms with statistical tests	Centralized	No	No	High
Grass et al. [11]	Anomaly detection for predictive maintenance	Historical sensor data and clustering algorithms	Centralized	No	No	High
Jahromi et al. [13]	Two-level ensemble attack detection and attribution framework	Decision tree and deep representation-learning model	Centralized	No	Yes	High
Know et al. [16]	Intrusion detection system for IEEE 1815.1-based power system	Bidirectional recurrent neural network	Centralized	No	Yes	High
Reza et al. [12]	Bolstering security of CPS in dc microgrids	NARX neural network	Centralized	No	Yes	High
Keshk et al. [14]	Privacy preservation in CPS	Naive Bayes, Decision Tree, Support Vector Machine	Centralized	Yes	No	High
Merabet et al. [21]	Dynamic adaptation in IoT Systems	Auto-NF framework	Centralized	No	No	High

properties of these clusters like the mean can give insight into their interpretation.

In [31], physical data machine learning approaches are developed and integrated for detecting Cyber Physical attacks in Cyber Manufacturing system. Authors developed two examples with simulation and experimentation to test and demonstrate the physical data machine learning security approach. In the 3D printing process example, they used vision as physical data source for machine learning. Three different machine learning algorithms are implemented with image classification. The anomaly detection method returned the highest accuracy of 96.1% in detecting a malicious defect in printing process.

Authors in [5] provide a self-adaptive and scalable prediction/detection mechanism for CPS. They propose a framework called AAPF-CPS, which combines several machine learning algorithms with statistical tests. With multiple classification algorithms, AAPF-CPS analyzes CPS network logs simultaneously and in real-time. Friedman's test is also used to rank each classifier for each context in AAPF-CPS. In [11], authors present an approach for anomaly detection regarding predictive maintenance in an CPS. In particular, they are focusing on historical sensor data from a real reflow oven that is used for soldering surface mount electronic components to printed circuit boards. The sensor data comprises information about the heat and the power consumption of individual fans inside a reflow oven. The data set contains time annotated sensor measurements in combination with additional process information over a period of more than seven years.

Authors in [13] propose a two-level ensemble attack detection and attribution framework specifically designed for CPS, particularly in industrial control systems. Their approach utilizes a decision tree combined with a novel deep representation-learning model at the first level for anomaly detection in imbalanced industrial control systems environments. This is followed by an ensemble deep neural network at the second level for attack attribution. Evaluation using real-world data from gas pipelines and water treatment systems demonstrates their model's effectiveness in outperforming other approaches with similar computational complexity.

In [23], the authors address the growing vulnerability of CPS, particularly in critical infrastructures such as water distribution systems (WDS), to cyber-attacks. In their proposed approach, the authors leverage fog computing and employ fuzzy logic-based methods, specifically the Intuitionistic Fuzzy System, for feature selection. Furthermore, a voting classifier is applied to the selected features to detect anomalies within the WDS. The effectiveness of their system is demonstrated through testing on the Water Distribution dataset, and the results are compared with state-of-the-art methods. This work contributes valuable

insights into enhancing the security of CPS, especially in critical infrastructure scenarios.

In [16], the authors propose an intrusion detection system tailored for an IEEE 1815.1-based power system integrated with CPS. With the integration of CPS into power systems, new communication requirements emerge, necessitating a mapping between existing distributed network protocol networks and IEC 61850 networks, as defined by the IEEE 1815.1 standard. Addressing the evolving threat landscape of advanced CPS cyberattacks, the study conducts a comprehensive analysis of the IEEE 1815.1-based power system network. It identifies an optimal application method for an intrusion detection system and introduces a bidirectional recurrent neural network-based anomaly detection system tailored for the nuances of an IEEE 1815.1-based network.

In [12], the authors focus on bolstering the security of CPS, with a specific emphasis on direct current microgrids. Addressing the rising concern of cyber threats in diverse applications like renewable energy and power distribution, they propose an innovative AI-driven method. This method, employing time-series analysis and a tailored nonlinear auto-regressive exogenous model (NARX) neural network, aims at timely detection and identification of cyber-attacks within dc microgrids. Notably, the strategy accommodates the challenges posed by false data injection attacks targeting voltage and current sensors. Through meticulous phases, including a cyber-attack-free data collection phase and subsequent online deployment of trained NARX networks, the proposed method showcases effectiveness.

Author in [14] propose an approach to address the challenge of privacy preservation in CPS, which generate vast amounts of big data from the integration of physical and digital entities. This paper explores the application of Independent Component Analysis (ICA) as a technique for protecting sensitive information from unauthorized access. The ICA method is employed to transform raw CPS data into a new configuration while retaining its data utility. The effectiveness of this mechanism is assessed using a power CPS dataset, demonstrating superior privacy protection compared to four other privacy-preservation techniques. Furthermore, the data utility is evaluated through the utilization of three machine learning algorithms (Naive Bayes, Decision Tree, and Support Vector Machine) to gauge their effectiveness in identifying normal and attack patterns before and after transformation. The results highlight the enhanced privacy protection and utility preservation achieved through the proposed ICA-based approach.

Authors in [21] propose a novel contribution to the dynamic adaptation of service-oriented systems within the context of the Internet of Things (IoT). The work addresses the increasing complexity of modern software systems and the demand for flexibility and self-adaptation in service-based IoT applications. The paper underscores the importance of predicting quality-of-service values, especially in scenarios where users interact with a limited subset of services, resulting in missing quality-of-service data. In response, the authors introduce the Auto-NF framework, which consists of three steps. The framework begins by proposing an extended similarity computation method, employing Euclidean distance to evaluate user similarity and identify comparable neighbors. The second step involves the formation of clusters of similar neighbors and subsequent partitioning of the initial matrix to alleviate data sparsity challenges. The third step introduces a straightforward neural network autoencoder, designed to learn deep features and select optimal latent factors to mitigate overfitting.

While the works in the Table 1 exhibit significant efforts in enhancing security and resilience in CPS, some common limitations can be identified. Most prominently, the majority of the approaches rely on centralized ML solutions, concentrating on global data processing and model training. This centralized paradigm presents challenges related to data privacy, security, and potential single points of failure. Additionally, centralized data storage raises scalability challenges as CPS systems grow in complexity. Moreover, continuous data transmission to a central server for model training and updates creates significant communication overhead, straining network resources, increasing vulnerability to interception, and potentially compromising real-time performance.

In contrast to the centralized approaches discussed earlier, decentralized methodologies, such as Federated Machine Learning (FML), emerge as promising alternatives that address some of the drawbacks associated with centralized models. Decentralized architectures, like FML, distribute the model training process across local devices or servers, eliminating the need for continuous data transmission to a central repository. This not only mitigates communication overhead but also safeguards against potential security and privacy concerns associated with centralized data storage. The decentralized nature of FML aligns with the distributed architecture of CPS, offering scalability as systems expand in complexity. Moreover, local training on diverse datasets allows for adaptation to specific environmental conditions, promoting a more resilient and adaptable CPS framework.

Before delving into the details of our FML-based architecture, it is essential to provide a brief overview of our earlier research contributions. These foundational elements, including our meta-model and multi-layer architecture for CPS, have paved the way for the development of our resilient CPS solution. By examining these pre-

existing components, we can better explain the evolutionary way that has led us to our current approach.

3 An overview of the proposed approach

3.1 Proposal of a CPS meta-model

Fig. 1 depicts the meta-model we have developed for addressing resilient CPS. The proposed meta-model is based on prior work about CPS and IoT [6, 8] and those related to designing and managing of self-adaptive systems [26]. Our model is composed of two parts: the first part includes all the generic elements of a CPS, whereas the second one holds the basic elements implementing the self-adaptation system. A CPS object is the composition of a virtual entity and a physical entity that is associated with, in order to highlight the fact that these two concepts belong together. The both present a state that may be perceived and changed during system running. Equation 1 states that a CPS is a non-empty set composed of intelligent components. The CPS is considered as the sum of these components, denoted as $comp_i$, where i ranges from 1 to n. The set notation implies that the CPS is formed by collecting and aggregating these components, showcasing the collaborative and interconnected nature of the system. Each

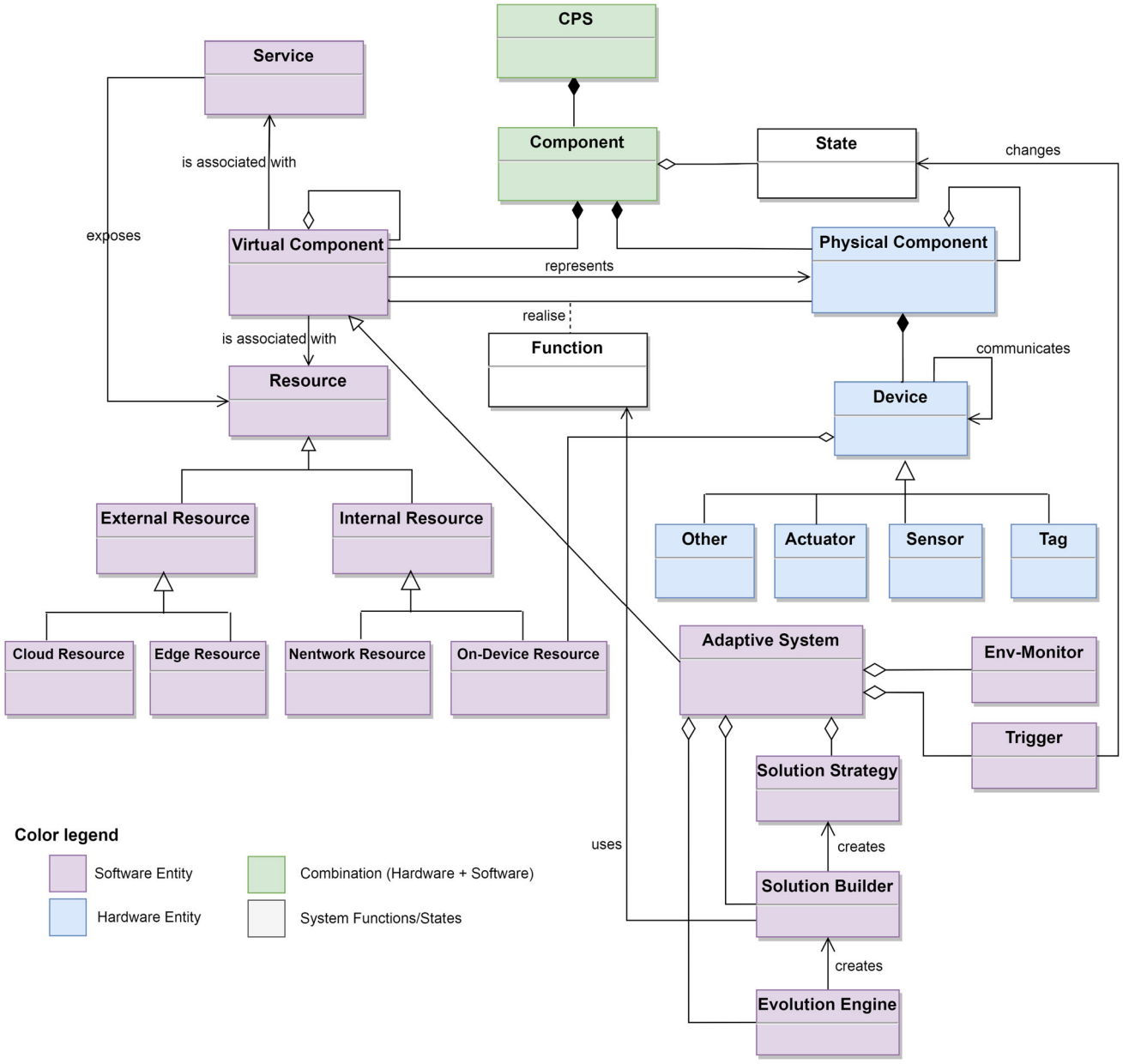


Fig. 1 A meta-model for CPS

intelligent component contributes to the overall functionality and intelligence of the CPS, and their interactions play a vital role in achieving the system's goals and objectives.

$$CPS = \sum_{i=1}^{n} (comp_i) \tag{1}$$

In fact, a component *comp* of a CPS is the intersection between what is considered physical (PhyC) and what is virtual (VrtC). It is no longer enough to understand, develop, manage and maintain physical and virtual components separately. We will therefore note:

$$comp = PhyC \cap VrtC \tag{2}$$

Where:

$$PhyC = \sum_{i=1}^{m} (P_i) \tag{3}$$

and

$$VrtC = \sum_{i=1}^{k} (V_i) \tag{4}$$

Equation 2 defines a component (*comp*) of a CPS as the intersection between the set of physical entities (*PhyC*) and the set of virtual entities (*VrtC*), where *comp* represents the shared functionalities between the physical and virtual aspects of the system. *PhyC* and *VrtC* are further elaborated in Eqs. 3 and 4, respectively. *PhyC* is defined as the sum of physical components P_i with i ranging from 1 to m, encompassing entities such as sensors, actuators, tags, and other physical devices. Similarly, *VrtC* is expressed as the sum of virtual components V_i with i ranging from 1 to k, capturing various virtual entities in the CPS. Together, these equations establish the composition of a CPS component by combining its physical and virtual elements.

The result of the intersection between PhyC and VrtC gives the set of functions F performed by the CPS, where:

$$\forall P_i \in PhyC, \exists V_j \in VrtC, \exists f_k \in F : V_j \wedge P_i \Rightarrow f_k \tag{5}$$

Formula 5 expresses the functional relationship within a CPS by stating that for every physical component (P_i) in the set of physical entities (*PhyC*), there exists a virtual component (V_j) in the set of virtual entities (*VrtC*), and there exists a function (f_k) in the set of functions (F). The logical condition $V_j \wedge P_i \Rightarrow f_k$ signifies that the combination of a specific virtual entity V_j and a corresponding physical entity P_i implies the execution of a particular function f_k. In simpler terms, this formula formalizes the notion that the interaction between physical and virtual components in a CPS results in the performance of specific functions, contributing to the system's overall functionality.

Physicals Components (PhyC) are the recognizable part of the physical environment that interests the user to achieve his goal. They can be any physical device such as: robot arm, cars, storage elements or supply chain item, computers, electronics, ect. Like presented in formula 3, in our CPS model, PhyC is a set of physical devices:

$$P_i \in \{Sensor, Actuator, Tag, Other\} \tag{6}$$

A virtual entity (VrtC), representing the physical entity in the digital domain, maintains a one-to-one association with its physical counterpart. This digital representation serves as a key linchpin within our meta-model, encapsulating not only the essential features of the physical components but also acting as the focal point for enabling dynamic self-awareness, self-learning, and self-reconfiguring capabilities in CPS. Beyond mirroring the physical components, the virtual entity integrates a sophisticated layer of software, encompassing resources, services, and mechanisms dedicated to facilitating seamless self-adaptation. This holistic approach ensures that the virtual entity not only captures the state of the physical entity but also enhances it in the digital realm, contributing to the overall intelligence and resilience of the CPS.

Resources are software components that provide data or used in the operation of physical entities. In our model, we identify two types of resources: Internal and external resource. Concerning the first type, there is a distinction between resources on the device and network resources. As the name suggests, on-device resources are hosted on the devices. It is the software deployed on the device (sensor and actuator) associated with the physical entity.

On the other hand, network resources are locally deployed (in the LAN of the same organization), include executable code for accessing, processing, and storing sensor information, as well as the code for controlling actuators. External resources are the resources that are available somewhere in the Cloud/Edge, for example, a back-end or a Cloud-based database. As we mentioned above, our meta-model includes all the elements of self-adaptive systems which are the key enablers of smart environments such as CPSs. In such environments, the system is composed of the necessary elements to know the environment and to act on it. Thus, the system has Triggers (see Fig. 1) for changing the state of the environment. It has, also, perceptors (Environment Monitor) for obtaining the environment current state. In a CPS, adaptation is the higher level of a smart system that is able to inspect itself, learning from experience and self-modifying its specification. Consequently, in our meta-model we have the Evolution Engine that is qualified to know the model of the system at run-time, and to change or influence the solution strategy built at design time. In this case, the system is able to revise its run-time model, thus producing a new solution

strategy. The Evolution Engine is able to set up a Solution Builder in which its role is to provide a completely new solution strategy by using a repository of established functionalities.

3.2 Proposal of an architecture for CPS

CPS are distributed and loosely coupled to cyber systems and physical systems that are monitored/controlled by user-defined semantic laws [17]. It is important to define the architecture of this paradigm as well as its components and their functionalities besides the used technology. The general objective of a CPS is to collect multiple data from various sources such as the data received from the sensors installed in the different production machines. Thereafter, convert them into information in the cyber world in order to process, to understand them, and then turn them into appropriate actions in the physical world. Therefore, our architecture [4] contains the main software modules that will ensure CPS functionalities such as: resource/data management, process planning, and the different modules which convert machines to be self-aware, self-learning and self-reconfiguring. As shown in Fig. 2, the proposed structure consists of the following layers:

Physical layer:

It represents the basic local assembly of machines connected to the Internet via the OPC-UA (OPen Connectivity - Unified Architecture) protocol to ensure communication standardization between different units. This layer consists mainly of sensors and actuators:

- Sensors are used to collect signals from machines and then transform them.
- Whereas, the actuators take electrical signals and combine them with a source of energy to create a physical movement.

Data/resource processing layer:

This layer collects data from various sources, and then it interacts directly with the data producer at the physical layer and the data storage in the Edge/Cloud. It contains the following modules: the conditioner, the data management, the resource management, the planning process, the monitoring and control module, the request management module and request management module.

Data storage layer:

This layer serves as the storage base for the CPS and it is distributed between the Edge and the Cloud, and this depends on the nature of the data and the time required to process it.

Learning application layer:

This layer ensures different functionalities such as: Resource allocation, QoS analysis, Process optimisation, Predictive maintenance and Fault detection. Within this layer, a variety of ML techniques, including regression, classification, clustering, and reinforcement learning, take center stage. These models are meticulously trained on a rich dataset, combining historical knowledge with real-time information, culminating in the ability to make predictions and informed decisions that drive the system's adaptive capabilities.

3.3 Using machine learning for faults prediction

To demonstrate the viability and effectiveness of this layer, we have implemented it in a practical context. This application aligns with the realm of: Failure Prediction Using Supervised and Unsupervised Learning Algorithms [4]. In this real-world scenario, our system leveraged the power of supervised and unsupervised learning algorithms to predict failures in a dynamic environment such as CPS. The main goal of the Fault Prediction module is to predict whether a device is exposed to failures or not. It allows, also, to identify the failure type. Therefore, its architecture will make use of supervised and unsupervised learning algorithms. First, we use classification techniques for separating data into two classes: faulty and correct data. Next, we apply the clustering method to the faulty data in order to group them into many clusters depending on fault type. Finally, we apply prediction.

In [4], we have detailed the different ML algorithms used for the implementation of the principle of fault prediction in the CPS. In our application, two ML algorithms are employed: Random Forest for classification, used to detect erroneous data, and K-Means for clustering, applied to identify fault types. Experiments have demonstrated the effectiveness of our approach in fault prediction, with an achieved accuracy surpassing 95%.

4 Towards a privacy-enhanced architecture empowered by FML

Our initial, centralized learning-based architecture [4] demonstrated considerable accuracy and precision in fault prediction. However, it also faced limitations regarding data security, privacy concerns, and communication overhead. These limitations highlighted the need for an alternative approach that could maintain the benefits of accurate predictions while addressing the challenges associated with centralized data collection and processing.

We propose a novel architecture empowered by FML to address these limitations. This FML-based approach distributes processing and intelligence to the edge of the network, enabling local data analysis and decision-making.

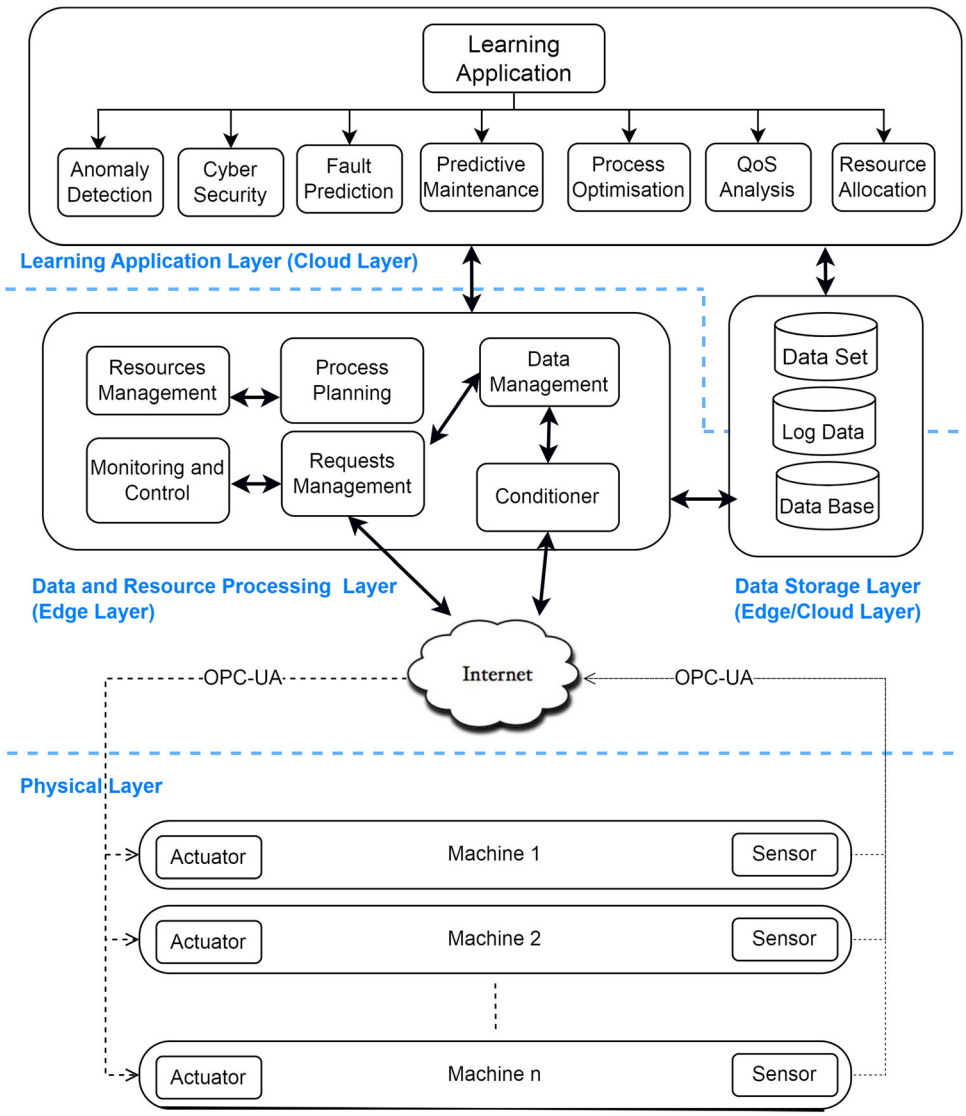


Fig. 2 The proposed architecture

By using FML, our architecture collaborates across edge devices for training machine learning models without centralized data collection. This decentralized approach preserves data privacy, reduces communication overhead, and enhances system resilience, all while upholding the high accuracy and precision obtained in the initial centralized architecture.

This section provides an overview of the proposed FML-based architecture, focusing on the components of the architecture and how they function together. We will only present what we have added compared to our previous architecture in terms of FML. As shown in Fig. 3, the architecture consists of three main entities: local collectors, Edge aggregators and a Cloud layer.

4.1 Architecture components

Local collectors:

Local Collectors are responsible for collecting and managing data from the local devices or nodes within a particular group or federation. They can encompass a wide range of devices, including industrial equipment, individual smartphones, IoT devices, sensors, and other endpoints. The Local Collectors ensure that data remains secure and private, often by aggregating or summarizing the data before its sent to the aggregator. The primary focus of Local Collectors is to prepare and send relevant information to the aggregator, respecting privacy and security constraints.

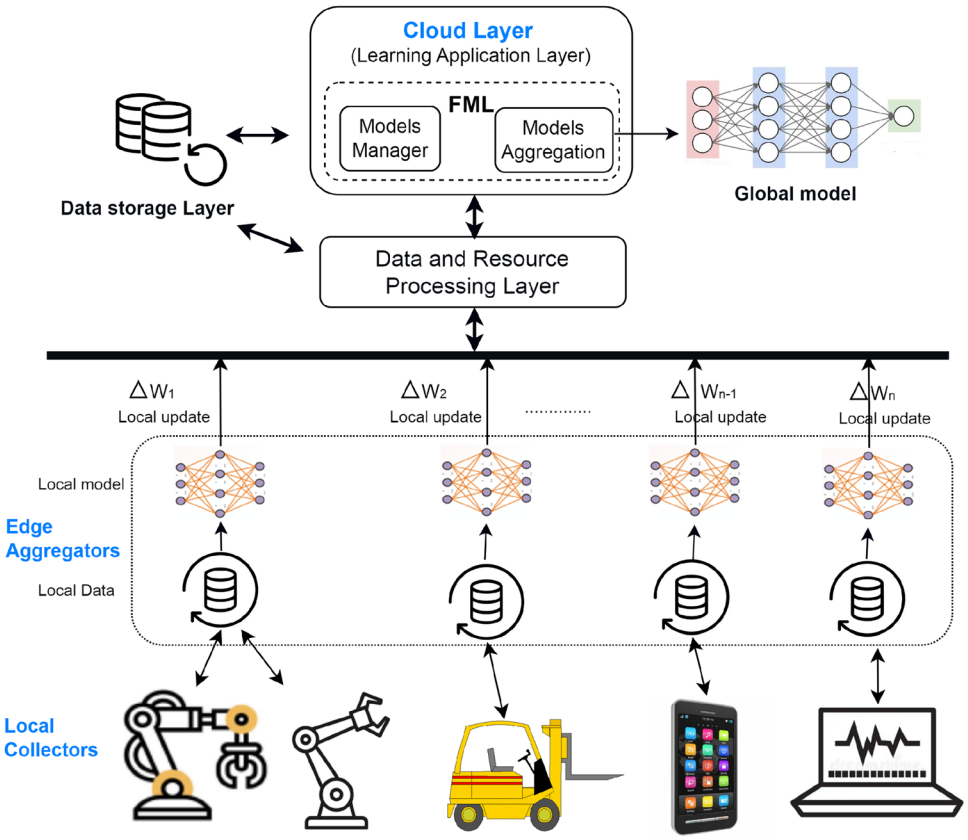


Fig. 3 The proposed FML-based architecture

Edge aggregators:

Edge Aggregators are responsible for aggregating the locally computed model updates from devices or nodes within their own federation. Each device trains its model locally using its own data, and then these local models are sent to the Edge Aggregator. It combines the models in some way, often by averaging the model parameters, to generate a global model update. This global model update is then shared with other federated groups or used to update the global model maintained by the Cloud layer. Edge aggregators facilitate collaborative learning across devices while ensuring that sensitive data remains on the local devices

Cloud layer:

It plays a crucial role in the FML framework. Its main goal is to facilitate the aggregation of knowledge that originates from various local aggregators. Due to privacy considerations, our architecture shares only the learned data that is coordinated by the Cloud layer, while the private or sensitive data are stayed at the local aggregators. Due to potential variations in data types and machine learning models across local aggregators, the Models Manager module in our architecture assumes the role of overseeing and harmonizing these diverse models, ensuring effective management and coordination. The updating of the global model is achieved through a synchronization process that involves both the global model and the local aggregators. This synchronization is achieved by the Models Aggregation module, which manages the coordination between these components.

4.2 Mode of operation

In our architecture, the goal is to collaboratively train a global model using data from N clients, each having their own local dataset denoted as D_i where: $1 \leqslant i \leqslant n$. This dataset consists of a set of K_i labeled pairs:

$$D_i = (X_i^{k_i}, Y_i^{k_i})_{k=1}^{k_i} \tag{7}$$

Where $X_i^{k_i}$ is referred to the training input, and $Y_i^{k_i}$ is the corresponding label.

In our architecture, the process revolves around collaborative training between Edge (Client side) and a Cloud layer (Server side). The goal is to update a global model by utilizing the local models trained on each client's local data. Algorithm 1 captures the behaviour of each client in our architecture.

Algorithm 1 Client Side (Edge layer)

Require: M_0 : initial global model
D : Local data
Ensure: the gradients of the local loss
1: $M_{local} \leftarrow M_0$
2: $M_{local} \leftarrow TrainLocalModel(M_{local}, D)$
3: $gradientsLocal \leftarrow ComputeGradients(M_{local}, D)$
4: SendToServer(gradientsLocal)

The client-side process begins with the initialization of a local model (M_{local}) using the initial global model (M_0). This local model is provided by the Cloud layer and it serves as the starting point for the client's training process (line 1). Subsequently, the local model is trained using the client's local data (line 2). Through this local training, a client trains its local model M_{local} using its local dataset D to minimize a local loss function: $L_i(M_{local})$. This is done by the formula:

$$M_{local} = argmin_M L_i(M_0) \tag{8}$$

Following the training phase, the gradients of the local loss with respect to the model's parameters are computed (line 3). These gradients are computed based on the local data and the current model parameters.

$$\nabla L_i = \nabla Loss(M_{local}, X_i, Y_i) \tag{9}$$

Finally, the client sends this gradient to the Cloud layer for aggregation and updating the global model (line 4). We note that clients send only the gradients of their local model parameters to the Cloud layer, rather than sending the entire model or raw data. This approach is essential to maintaining data privacy and reducing communication overhead.

Regarding the server side, its process is described in Algorithm 2. The server plays a pivotal role in managing the aggregation and updates of the global model by utilizing contributions from the devices participating in the training. To begin, the server initializes the global model denoted as M_{global} (line 1). This model serves as the starting point for the collaborative training process. Then, the algorithm proceeds by iterating through a predetermined number of rounds, indicated by the variable *nbrRound* (line 2). Within each round, the server performs several tasks. It first distributes the current global model M_{global} to the participating clients (line 3). Then, it sets up an accumulator named *aggregated_gradients* to gather the gradients sent by devices (line 4). After that and for each device enlisted in the list of participating devices, the server receives the gradients (*gradients_local*) transmitted by the device. The server subsequently aggregates these local gradients by adding them to the *aggregated_gradients* accumulator (lines 4–8).Once all the gradients have been aggregated, the server calculates the averaged gradients by dividing the accumulated *aggregated_gradients* by the total number of participating devices (line 9).Afterwards, the server employs the calculated averaged gradients to update the global model (M_{global}) through the *UpdateGlobalModel* function (line 10).

The global model at iteration t+1, denoted as $M_{global_{t+1}}$, is obtained by aggregating the local models from the participating clients. In our architecture the aggregation is done using averaging method (where N is the number participating clients):

$$M_{global_{t+1}} = \frac{1}{N}\sum_{i=1}^{N} M^i_{global_t} \tag{10}$$

We note that $M_{global_{t+1}}$ is used as the new global model for the next iteration.

Algorithm 2 Server Side (Cloud layer)

Require: Devices: the participating devices for training
nbrRound : positiveNumber
Ensure: an updated global model
1: $M_{global} \leftarrow InitializeGlobalModel()$
2: **for** $(round \leftarrow 0$ to $nbrRound - 1)$ **do**
3: broadcast M_{global} to all clients in Devices
4: $aggregated_gradients \leftarrow 0$
5: **for** (each device in Devices) **do**
6: $gradients_local \leftarrow ReceiveGradientsFromClient()$
7: $aggregated_gradients \leftarrow aggregated_gradients + gradients_local$
8: **end for**
9: $averaged_gradients \leftarrow aggregated_gradients/nbr_devices$
10: $M_{global} \leftarrow UpdateGlobalModel(M_{global}, averaged_gradients)$
11: **end for**
12: return (M_{global})

5 Implementation and experiments

5.1 Some implementation aspects

In this section, we explain the experimental setup we used to implement our FML-based architecture. The TensorFlow Federated (TFF) [29] framework is employed for parallel client training, and the training process involves data batch division and distribution to individual clients. The federated model is constructed based on the same architecture as the global model and is trained using the Federated Averaging (FedAvg) algorithm [20].

As we mentioned previously, the training process centers on a collaborative training task involving both Edge Collectors and a Cloud layer. The goal is to update a global model by utilizing the local models trained on each client's local data. The first step of the training process is the initialization of the global model. This step sets the model architecture and parameter values before engaging in collaborative training. It involves setting up the architecture of the learning model (specifying the number and types of layers, activation functions, and other architectural components). In our case, we define the architecture of the global model using TensorFlow'sKeras API within TFF's context. We use the *tff.learning.from_keras_model* function to convert the Keras model into a TFF-compatible federated learning model.

For clients selection, we use the TFF's built-in federated computations and functions. The *tff.federated* module provides functionalities for creating federated computations, including the selection of clients for each round of federated training. In our architecture, the *select_clients* function is defined as a federated computation using the *@tff.federated_computation decorator*. The function takes *client_weights* as an argument, which could be a federated integer type representing the weights of each client.

Regarding the step of local training (client update) we define the function *local_training* as follow: $local_training(global_model, data_batch)$. The function takes as parameters the *global_model* as the model architecture and a $data_batch$ representing the local data on the client. This function allows as initializing a copy from the global model using *tff.learning.ModelWeights.from_model*. Local training is performed using TFF's gradient tape mechanism. The loss is computed based on the model's predictions and actual labels, and gradients are calculated. After executing the federated computation, the updated local model will contain the local model with updated parameters after the local training process. The resulting local model updates are sent to the Cloud layer.

For model aggregation we have used the Federated Averaging (FedAvg) algorithm. It permits aggregation local model updates from clients to update the global model. In our case, we define the function *models_aggregation* as a federated computation using the *@tff.federated_computation* decorator as follow: $models_aggregation(global_model, local_models)$. The function takes as an input the global model and a list of local models representing the updated models from selected clients. In this function, the predefined *with_weights* method is used to attach the client weights to each local model. Also, the predefined *tff.federated_mean* method is applied to compute the weighted average of the local models. This function calculates the weighted sum of model weights and divides it by the total weight to obtain the aggregated weights.

After the global model has been updated through the aggregation process, it needs to be broadcasted to all participating clients. This synchronization guarantees that each client has access to the most up-to-date global model for the subsequent round of training.

5.2 Experimental evaluation

5.2.1 Data preparation

Data Preparation Data collection and preparation are fundamental steps that influence the entire ML pipeline, and in FML, they become even more critical due to the distributed and privacy-sensitive nature of the data. Properly preparing and managing data across distributed clients sets the foundation for successful federated learning. In this experiment, we used the same dataset as in our previous architecture. This allows us to compare the two obtained results. The data is collected from *Kaggle* known as predictive maintenance which offers a synthetic data set [19]. It reflects the actual predictive maintenance data of a system encountered in the industry. The dataset consists of

Table 2 Data set structure

Column	Data type
UDI	Int64
Product ID	Object
Type	Object
Air temperature [K]	Float64
Process temperature [K]	Float64
Rotational speed [rpm]	Int64
Torque [Nm]	Float64
Tool wear [min]	Int64
Target	Int64
Failure Type	Object

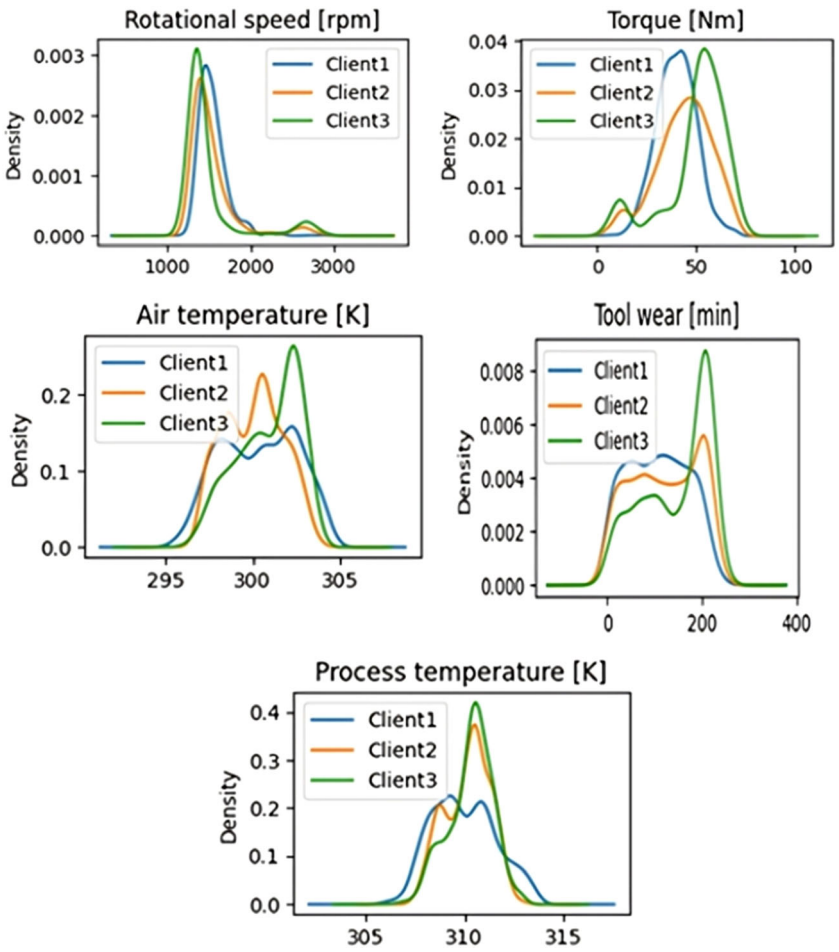


Fig. 4 Data visualization

10,000 lines and Table 2 presents the different columns of this dataset.

In our approach the most important stages of data preparation is density estimation which refers to the process of estimating the underlying probability distribution of data within each client's local dataset without aggregating the actual data itself. This estimation is performed locally on individual clients, and the results are then used to guide the training process. In this step, we conducted initial experimentation to analyze and study the data from our clients. With the aim of evaluating our proposal, we partitioned the dataset across three distinct clients. This partitioning was carried out over multiple iterative phases, the goal being to achieve a reasonable correlation among the datasets of the various clients.To facilitate this analysis, we used data visualization methods, including correlation matrix plot. This allowed us to visually observe the relationships between different variables and highlight emerging patterns and trends. Figure 4 presents the data visualization of the three clients.

Figure 4 shows a clear correlation among the data of the various clients, particularly in relation to the variables Rotational speed, Torque, and Tool wear. A significant correlation in density estimation suggests that the data distributions across clients are aligned. This can simplify model training, aggregation, and result in faster convergence.

5.2.2 Training process

As previously discussed, the training process relies on collaboration between the server and the clients. This collaborative process iterates over multiple rounds with the objective of achieving a minimal loss value and a high degree of accuracy. As demonstrated in Algorithm 2, local training relies on calculating the loss value, which is a measure of how well the model's predictions align with the actual target values in the training data. A smaller loss value means that the model's predictions are closer to the actual data points.

In this experimentation, the global DL network is formulated as a Sequential model. Its architecture consists of two densely connected layers with ReLU (Rectified Linear Unit) activation functions, followed by a final layer with a sigmoid activation function. The global DL network is trained using the FedAvg algorithm [20], where model

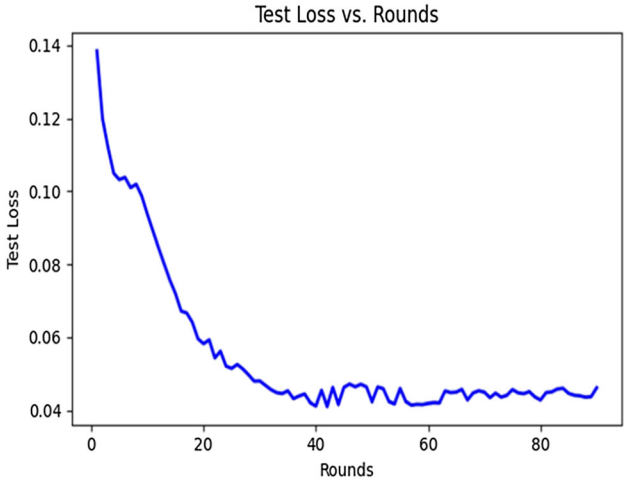


Fig. 5 Loss values vs number of rounds

updates from local clients are aggregated on the Cloud layer. Each training round consists of a set number of epochs (num_epochs = 70), with a specified batch size (batch_size = 32) for processing data. Additionally, the entire training process spans a predetermined number of rounds (num_rounds = 80).

Figure 5 shows the loss average during local optimization. We observe that the number of training rounds can significantly affect the loss value. In the initial rounds of training, we observe an important reduction in the loss value. This is because the model is learning from the client data and making significant improvements. After a sufficient number of rounds (round = 40), the loss value is stabilizing at a certain minimum value. This minimum represents the best achievable performance given the data and model architecture. Further rounds might not significantly reduce the loss. We tried to increase the number of rounds, but we noticed that the loss value is constant and sometimes increasing, and this indicates that the model has reached a point of convergence. Convergence means that the model has learned most of the patterns and relationships present in the training data, and further training does not significantly improve performance.

Another important aspect that warrants attention when training local models is the occurrence of Overfitting, which mean that the model learns the training data too well. In this case, the model might become too specialized in capturing noise in the training data, leading to poor generalization on unseen data.

Figure 6 provides a clear depiction of how accuracy is related to the number of training rounds. We observe that accuracy increases until it reaches a fixed or stable value, it typically indicates that the model has converged and it has reached its optimal performance on the given data. This stable accuracy value is positive because it indicates that the model has not overfit the training data by memorizing noise or irrelevant details.

As a result of this experimentation, it is essential to strike a balance between training long enough for the model to learn but not so long that it starts to overfit or reaches a point of diminishing returns in terms of improvement. In fact, monitoring both training and validation accuracy is crucial to make informed decisions about when to stop training and to ensure the model achieves the best overall performance. We note that in this experimentation we gave great attention to preparing data while using FML-based techniques to train local models in order to obtain high accuracy, so that these results allow us to obtain good expectations with the new data (unseen data). Among the techniques used during training:

- Client-Level Regularization: We apply regularization techniques like Federated L1 and Federated L2 on the client side to encourage simpler local models.
- Early Stopping: Monitor the performance of the global model on a validation dataset during training rounds to detect when the model starts to overfit.
- Learning Rate: Learning rate controls the step size during gradient descent. Tuning it helps find the right balance between convergence speed and model accuracy.

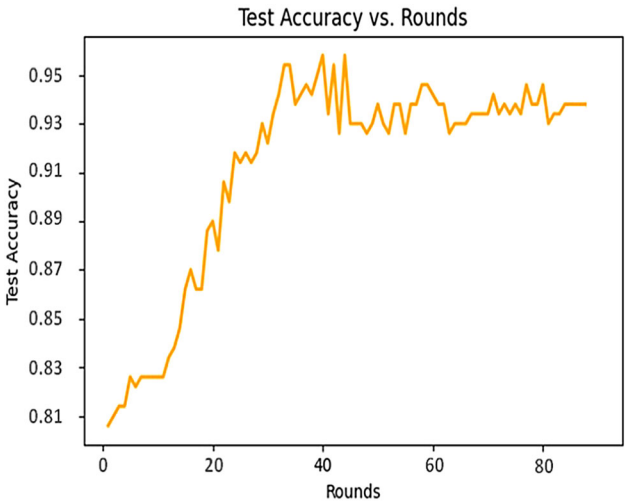


Fig. 6 Accuracy vs number of rounds

Table 3 Federated vs Centralized ML

	Recall	Accuracy	Precision
Federated	0.9584	0.9544	0.9420
Centralized	0.9580	0.9785	0.9695

5.2.3 Discussion

In comparing federated learning andcentralized learning based on the provided metrics (see Table 3), we observe the following results: federated learning achieves an accuracy of 0.95, a precision of 0.94, and a recall of 0.95. In contrast, centralized learning demonstrates higher values, with an accuracy of 0.97, a precision of 0.96, and a recall of 0.95.

Regarding accuracy, centralized learning outperforms federated learning, indicating a better ability to make overall correct predictions on the data. In terms of precision, both approaches maintain high precision values, with centralized learning slightly surpassing federated learning. This suggests that centralized learning is slightly better at avoiding false positive predictions. However, both centralized learning and federated learning exhibit the same recall value of 0.95, indicating equal performance in correctly identifying true positives.

The trade-offs between these approaches stem from federated learning's training on decentralized data from multiple clients, potentially leading to a more challenging optimization problem. On the other hand, centralized learning benefits from a centralized, potentially more comprehensive and well-curated dataset, which may account for its higher accuracy and precision.

Centralized learning is favoured when access to a large, well-curated, and representative dataset is available. This approach is particularly advantageous for tasks where centralizing data is feasible and practical. However, it is essential to acknowledge that in many real-world scenarios, especially in CPS, centralizing data may not always be a viable or desirable option. CPS systems often involve distributed and interconnected devices or sensors that generate data in real-time. Centralized data management in systems like CPS faces various challenges, including:

- Data Privacy and Security: Many CPS applications involve sensitive data, and centralizing it might raise significant privacy and security concerns.
- Data Localization: Data is generated and processed at the edge, near the source, and centralizing it can be technically challenging and resource-intensive.
- Latency Sensitivity: In CPS, low-latency processing is critical for timely decision-making and control. Centralizing data can introduce unacceptable delays.
- Scalability: CPS systems can involve a large number of devices, leading to scalability issues when trying to centralize data from all sources.

In these scenarios, FML become valuable. FML allows models to be trained locally on distributed data sources, preserving data privacy, reducing latency, and accommodating the distributed nature of CPS systems [18, 24].

In the context of our proposed FML-based architecture, it is essential to highlight the specific challenges and limitations associated with FML in CPS. While federated learning offers advantages, such as preserving data privacy and reducing latency, it is not without its challenges in the CPS domain. One significant challenge is the potential heterogeneity of local datasets across distributed CPS devices. The diversity in data types, formats, and quality may introduce additional complexities during the federated learning process. Model training on such heterogeneous datasets requires careful consideration to ensure the robustness and generalization of the learned models. Furthermore, the dynamic nature of CPS environments introduces challenges in maintaining model accuracy over time. As devices evolve or new devices join the network, ensuring the continuous adaptability and performance of the federated learning models becomes a non-trivial task. Another aspect to consider is the communication overhead involved in federated learning, particularly in CPS scenarios with resource-constrained devices. The exchange of model updates and gradients between devices and the central server may incur additional communication costs, impacting the overall system efficiency.

These challenges emphasize the need for tailored solutions and optimization strategies when implementing federated learning in CPS. As our architecture aims to address these challenges, future research directions may explore techniques for handling heterogeneous datasets, ensuring long-term model adaptability, and optimizing communication protocols for resource-constrained CPS devices.

6 Conclusion

The evolution of CPS from static, pre-defined solutions to self-adaptive entities is an imperative response to the ever-increasing complexity and dynamism of the modern world. As highlighted in this paper, the incorporation of ML algorithms, particularly within the framework of FML, represents a pivotal step towards achieving this adaptability. The ability of CPS to autonomously monitor their environment, assess changes, and dynamically adjust their behaviour not only enhances their efficiency but also ensures compliance with performance criteria in the face of unpredictable environmental variations. This paper has unveiled an innovative approach that leverages the power of FML to empower CPS with real-time self-adaptation capabilities. Building upon previous investigations into resilient CPS, the proposed multi-layer architectural serves as a blueprint for designing CPS that are self-aware, self-learning, and self-reconfiguring. Furthermore, our commitment to data privacy and security in the context of distributed data processing is a significant advancement.

As future work, we envision several directions for research and development:

- To effectively enhance the resilience in CPS, it's vital to rigorously compare our FML-based architecture, that use FedAvg algorithm, against established federated learning techniques like FedProx and FedDP. This analysis will provide valuable insights into the relative strengths and weaknesses of each technique, particularly in the context of enhancing CPS adaptability.
- Conducting more focused experiments: We aim to conduct specialized experiments explicitly designed to assess the security and privacy properties of our FML-based architecture. These focused experiments will provide empirical evidence and concrete insights into the effectiveness of our architecture in safeguarding data.
- Integration of privacy-preserving mechanisms: Beyond FML's inherent privacy benefits, we will strengthen security through exploring Privacy-Preserving Mechanisms like differential privacy, secure multi-party computation, and homomorphic encryption, ensuring data confidentiality and security during model training.

Acknowledgements The author would like to thank the anonymous reviewers for their valuable comments and suggestions, which were helpful in improving the paper.

References

1. Al Faruque, M., Regazzoni, F., Pajic, M.: Design methodologies for securing cyber-physical systems. In 2015 International Conference on Hardware/Software Codesign and System Synthesis (CODES+ ISSS), pages 30–36. IEEE, (2015)
2. Awad, M., Khanna, R.: Efficient learning machines: theories, concepts, and applications for engineers and system designers. Springer, Cham (2015)
3. Azeri, N., Hioual, O., Hioual, O.: Towards an approach for modeling and architecting of self-adaptive cyber-physical systems. In 2022 4th International Conference on Pattern Analysis and Intelligent Systems (PAIS), pages 1–7. IEEE, (2022)
4. Azeri, N., Zouikri, Z., Rezgui, M., Hioual, O., Hioual, O.: Fault prediction using supervised and unsupervised learning algorithms in cyber physical systems. In 2022 2nd International Conference on New Technologies of Information and Communication (NTIC), pages 1–6. IEEE, (2022)
5. Babouche, S., Ouchani, S., Zghal, M. et al.: An adaptive attack prediction framework in cyber-physical systems. In 2022 Ninth International Conference on Software Defined Systems (SDS), pages 1–7. IEEE, (2022)
6. Bauer, M., Bui, N., De Loof, J., Magerkurth, C., Nettsträter, A., Stefa, J., Walewski, J.W.: Iot reference model. Enabling things to talk: designing IoT solutions with the IoT architectural reference model, pp. 113–162, (2013)
7. Carvalho, T.P., Soares, F.A.A.M.N., Vita, R., Francisco, Rd.P., Basto, J.P., Alcalá, S.G.S.: A systematic literature review of machine learning methods applied to predictive maintenance. Comput. Indust. Eng. **137**, 106024 (2019)
8. Dafflon, B., Moalla, N., Ouzrout, Y.: The challenges, approaches, and used techniques of cps for manufacturing in industry 4.0: a literature review. Int. J. Adv. Manuf. Technol. **113**, 2395–2412 (2021)
9. Diaz-Rozo, Javier, Bielza, Concha, Larrañaga, Pedro: Machine learning-based cps for clustering high throughput machining cycle conditions. Procedia Manufacturing **10**, 997–1008 (2017)
10. Gerostathopoulos, Ilias, Skoda, Dominik, Plasil, Frantisek, Bures, Tomas, Knauss, Alessia: Tuning self-adaptation in cyber-physical systems through architectural homeostasis. J. Syst. Softw. **148**, 37–55 (2019)
11. Graß, A., Beecks, C., Soto, J.A.C.: Unsupervised anomaly detection in production lines. In Machine learning for cyber physical systems: selected papers from the International Conference ML4CPS 2018, pp. 18–25. Springer, (2019)
12. Habibi, M.R., Baghaee, H.R., Dragičević, T., Blaabjerg, F.: Detection of false data injection cyber-attacks in dc microgrids based on recurrent neural networks. IEEE J. Emerg. Selected Topics Power Electron. **9**(5), 5294–5310 (2020)
13. Jahromi, A.N., Karimipour, H., Dehghantanha, A., Choo, K.-K.R.: Toward detection and attribution of cyber-attacks in IoT-enabled cyber-physical systems. IEEE Internet Things J. **8**(17), 13712–13722 (2021)
14. Keshk, M., Moustafa, N., Sitnikova, E., Turnbull, B.: Privacy-preserving big data analytics for cyber-physical systems. Wireless Networks, pp. 1–9, (2022)
15. Khan, W.Z., Ahmed, E., Hakak, S., Yaqoob, I., Ahmed, A.: Edge computing: a survey. Future Gen. Comput. Syst. **97**, 219–235 (2019)
16. Kwon, S., Yoo, H., Shon, T.: Ieee 1815.1-based power system security with bidirectional rnn-based network anomalous attack detection for cyber-physical system. IEEE Access, 8:77572–77586, (2020)
17. Lesch, Veronika, Züfle, Marwin, Bauer, André, Iffländer, Lukas, Krupitzer, Christian, Kounev, Samuel: A literature review of IoT and cps-what they are, and what they are not. J. Syst. Softw. **200**, 111631 (2023)
18. Liu, Ji., Huang, Jizhou, Zhou, Yang, Li, Xuhong, Ji, Shilei, Xiong, Haoyi, Dou, Dejing: From distributed machine learning to federated learning: a survey. Know. Inform. Syst. **64**(4), 885–917 (2022)
19. Matzka, S.: Explainable artificial intelligence for predictive maintenance applications. In 2020 third international conference on artificial intelligence for industries (ai4i), pp. 69–74. IEEE, (2020)
20. McMahan, B., Moore, E., Ramage, D., Hampson, S., y Arcas, B.A.: Communication-efficient learning of deep networks from decentralized data. In Artificial intelligence and statistics, pp. 1273–1282. PMLR, (2017)
21. Merabet, F.Z., Benmerzoug, D.: Qos prediction for service selection and recommendation with a deep latent features autoencoder. Comput. Sci. Inform. Syst. **19**(2), 709–733 (2022)
22. Muccini, H., Sharaf, M., Weyns, D.: Self-adaptation for cyber-physical systems: a systematic literature review. In Proceedings of the 11th international symposium on software engineering for adaptive and self-managing systems, pp. 75–81, (2016)
23. Nayak, O., Lachure, J., Doriya, R.: Fog enabled cyber-physical attack detection using ensemble machine learning. In 2022 1st International Conference on Sustainable Technology for Power and Energy Systems (STPES), pp. 1–6. IEEE, (2022)
24. Papadopoulos, P., Abramson, W., Hall, A.J., Pitropakis, N., Buchanan, W.J.: Privacy and trust redefined in federated machine learning. Machine Learn. Knowl. Extr. **3**(2), 333–356 (2021)
25. Ryalat, M., ElMoaqet, H., AlFaouri, M.: Design of a smart factory based on cyber-physical systems and internet of things towards industry 4.0. Appl. Sci. **13**(4), 2156 (2023)

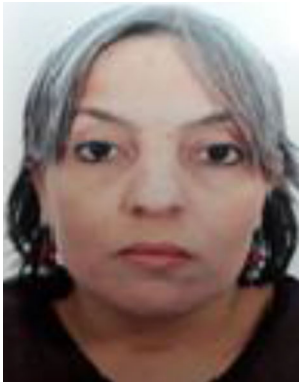
26. Sabatucci, L., Seidita, V., Cossentino, M.: The four types of self-adaptive systems: a metamodel. In Intelligent Interactive Multimedia Systems and Services 2017 10, pp. 440–450. Springer, (2018)
27. Samad, T.: Human-in-the-loop control and cyber–physical–human systems: applications and categorization. Cyber–Physical–Human Systems: Fundamentals and Applications, pp. 1–23, (2023)
28. Sunyaev, A., Sunyaev, A.: Cloud computing. Internet computing: principles of distributed systems and emerging internet-based technologies, pp. 195–236, (2020)
29. TFF. Tensorflow federated: Machine learning on decentralized data, (2023). https://www.tensorflow.org/federated
30. Wong, K.-S., Nguyen-Duc, M., Le-Huy, K., Ho-Tuan, L., Do-Danh, C., Le-Phuoc, D.: An empirical study of federated learning on iot-edge devices: Resource allocation and heterogeneity. arXiv preprint arXiv:2305.19831, (2023)
31. Wu, M., Song, Z., Moon, Y.B.: Detecting cyber-physical attacks in cybermanufacturing systems with machine learning methods. J. Intell. Manuf. **30**, 1111–1123 (2019)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Nabila Azeri is a PhD student at ICOSI Laboratory, Abbes Laghrour University - Khenchela (Algeria). She received her MSc degree in Computer Sciences from Constantine 2 University, Algeria, in 2015. Azeri's research interests span diverse domains, with a focus on Artificial Intelligence, Machine Learning, Cyber-Physical Systems, and Software Engineering.



Ouided Hioual received her BS degree in Computer Science from Mentouri University of Constantine (Algeria) in 2004, and MS degree in Computer Science from Abbes Laghrour University of Khenchela (Algeria) in 2009. Currently, she is working as Associate professor at the Department of Mathematics and Computer Science at Abbes Laghrour University of Khenchela (Algeria). She supervised many PhD, Master and License students Since September 20012 until Now. Since October 2010 until December 2019 she has prepared his PhD in Computer Science. She has published a number of articles in international conferences and journals. Her research interests include Semantic Web, Ontology, Web Services, Multi Agent System, Cloud Computing, Knowledge management, the reasoning in artificial intelligence, the engineering of the Human-machine interfaces and the automatic treatment of the natural language.



Ouassila Hioual received her BS and MS in Computer Science from the Mentouri University of Constantine, Algeria in 2002 and 2005. She has worked as a Lecturer at the Department of Computer Science and Mathematics Science at the Mentouri University of Constantine, Algeria from 2005 to 2008. Currently, she works as a full professor at the Department of Mathematics and Computer Science at Abbes Laghrour University of Khenchela, Algeria. She supervised many doctoral, master and license students. Since September 2006 until October 2011, she has prepared his PhD in Computer Science. She has published a number of articles in international conferences and journals. Her research interests include CPS, Industry 4.0, data sciences, IoT and Cloud Computing environments, energy consumption.